More Mathematical Library Functions

- Require `cmath` header file
- Take `double` as input, return a `double`
- Commonly used functions:

`sin`	Sine
`cos`	Cosine
`tan`	Tangent
`sqrt`	Square root
`log`	Natural (e) log
`abs`	Absolute value (takes and returns an int)

More Mathematical Library Functions

- These require `cstdlib` header file
- `rand()`: returns a random number (`int`) between `0` and the largest int the compute holds. Yields same sequence of numbers each time program is run.
- `srand(x)`: initializes random number generator with `unsigned int x`

Hand Tracing a Program

- Hand trace a program: act as if you are the computer, executing a program:
  - step through and 'execute' each statement, one-by-one
  - record the contents of variables after statement execution, using a hand trace chart (table)
- Useful to locate logic or mathematical errors

Program 3-27 with Hand Trace Chart

Program 3-27 **(with hand trace chart filled)**

```
1  // This program asks for three numbers, then
2  // displays the average of the numbers.
3  #include <iostream>
4  using namespace std;
5  int main()
6  {
7      double num1, num2, num3, avg;
8      cout << "Enter the first number: ";
9      cin >> num1;
10     cout << "Enter the second number: ";
11     cin >> num2;
12     cout << "Enter the third number: ";
13     cin >> num3;
14     avg = num1 + num2 + num3 / 3;
15     cout << "The average is " << avg << endl;
16     return 0;
17 }
```

num1	num2	num3	avg
?	?	?	?
?	?	?	?
10	?	?	?
10	?	?	?
10	20	?	?
10	20	?	?
10	20	30	?
10	20	30	40
10	20	30	40

Relational Operators

- Used to compare numbers to determine relative order
- Operators:

`>`	Greater than
`<`	Less than
`>=`	Greater than or equal to
`<=`	Less than or equal to
`==`	Equal to
`!=`	Not equal to

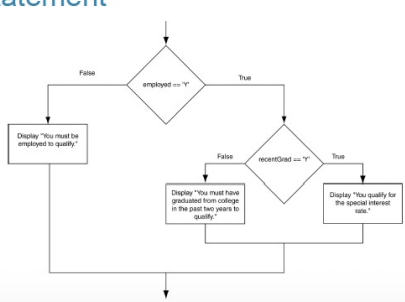
`if` Statement Notes

- Do not place `;` after `(`*`expression`*`)`
- Place *`statement`*`;` on a separate line after `(`*`expression`*`)`, indented:

```
if (score > 90)
    grade = 'A';
```

- Be careful testing `float`s and `double`s for equality
- `0` is `false`; any other value is `true`

Flowchart for a Nested `if` Statement



Flags

- Variable that signals a condition
- Usually implemented as a `bool` variable
- Can also be an integer
  - The value `0` is considered `false`
  - Any nonzero value is considered `true`
- As with other variables in functions, must be assigned an initial value before it is used

Logical Operators

- Used to create relational expressions from other relational expressions
- Operators, meaning, and explanation:

`&&`	AND	New relational expression is true if both expressions are true
`||`	OR	New relational expression is true if either expression is true
`!`	NOT	Reverses the value of an expression – true expression becomes false, and false becomes true

Logical Operator-Notes

- `!` has highest precedence, followed by `&&`, then `||`
- If the value of an expression can be determined by evaluating just the sub-expression on left side of a logical operator, then the sub-expression on the right side will not be evaluated (*short circuit evaluation*)

Menus

- Menu-driven program: program execution controlled by user selecting from a list of actions
- Menu: list of choices on the screen
- Menus can be implemented using `if/else if` statements

Menu-Driven Program Organization

- Display list of numbered or lettered choices for actions
- Prompt user to make selection
- Test user selection in *`expression`*
  - if a match, then execute code for action
  - if not, then go on to next *`expression`*

Validating User Input

- Input validation: inspecting input data to determine whether it is acceptable
- Bad output will be produced from bad input
- Can perform various tests:
  - Range
  - Reasonableness
  - Valid menu choice
  - Divide by zero

Input Validation in Program 4-19

```
16   int testScore;  // To hold a numeric test score
17
18   // Get the numeric test score.
19   cout << "Enter your numeric test score and I will\n"
20        << "tell you the letter grade you earned: ";
21   cin >> testScore;
22
23   // Validate the input and determine the grade.
24   if (testScore >= MIN_SCORE && testScore <= MAX_SCORE)
25   {
26      // Determine the letter grade.
27      if (testScore >= A_SCORE)
28        cout << "Your grade is A.\n";
29      else if (testScore >= B_SCORE)
30        cout << "Your grade is B.\n";
31      else if (testScore >= C_SCORE)
32        cout << "Your grade is C.\n";
33      else if (testScore >= D_SCORE)
34        cout << "Your grade is D.\n";
35      else
36        cout << "Your grade is F.\n";
37   }
38   else
39   {
40      // An invalid score was entered.
41      cout << "That is an invalid score. Run the program\n"
42           << "again and enter a value in the range of\n"
43           << MIN_SCORE << " through " << MAX_SCORE << ".\n";
44   }
```

Comparing Characters

- Characters are compared using their ASCII values
- 'A' < 'B'
  - The ASCII value of 'A' (65) is less than the ASCII value of 'B'(66)
- '1' < '2'
  - The ASCII value of '1' (49) is less than the ASCI value of '2' (50)
- Lowercase letters have higher ASCII codes than uppercase letters, so 'a' > 'Z'

Comparing `string` Objects

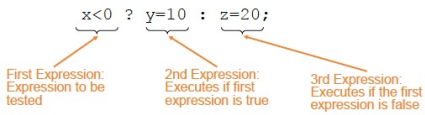
- Like characters, strings are compared using their ASCII values

```
string name1 = "Mary";
string name2 = "Mark";

name1 > name2   // true
name1 <= name2 // false
name1 != name2  // true

name1 < "Mary Jane" // true
```

The characters in each string must match before they are equal

The Conditional Operator

- Can use to create short `if/else` statements
- Format: `expr ? expr : expr;`



The Conditional Operator

- The value of a conditional expression is
  - The value of the second expression if the first expression is true
  - The value of the third expression if the first expression is false
- Parentheses `()` may be needed in an expression due to precedence of conditional operator

The `switch` Statement in Program 4-23

Program 4-23

```
// The switch statement in this program tells the user something
// he or she already knows: the data just entered!
#include <iostream>
using namespace std;

int main()
{
   char choice;

   cout << "Enter A, B, or C: ";
   cin >> choice;
   switch (choice)
   {
      case 'A': cout << "You entered A.\n";
                break;
      case 'B': cout << "You entered B.\n";
                break;
      case 'C': cout << "You entered C.\n";
                break;
      default:  cout << "You did not enter A, B, or C!\n";
   }
   return 0;
}
```

Program Output with Example Input Shown in Bold

```
Enter A, B, or C: B [Enter]
You entered B.
```

Program Output with Example Input Shown in Bold

```
Enter A, B, or C: F [Enter]
You did not enter A, B, or C!
```

`switch` Statement Requirements

1) *`expression`* must be an integer variable or an expression that evaluates to an integer value
2) *`exp1`* through *`expn`* must be constant integer expressions or literals, and must be unique in the `switch` statement
3) `default` is optional but recommended

`switch` Statement-How it Works

1) *`expression`* is evaluated
2) The value of *`expression`* is compared against *`exp1`* through *`expn`*.
3) If *`expression`* matches value *`expi`*, the program branches to the statement following *`expi`* and continues to the end of the `switch`
4) If no matching value is found, the program branches to the statement after `default:`

`break` Statement

- Used to exit a `switch` statement
- If it is left out, the program "falls through" the remaining statements in the `switch` statement

Using `switch` in Menu Systems

- `switch` statement is a natural choice for menu-driven program:
  - display the menu
  - then, get the user's menu selection
  - use user input as `expression` in `switch` statement
  - use menu choices as *`expr`* in `case` statements

More About Blocks and Scope

- Scope of a variable is the block in which it is defined, from the point of definition to the end of the block
- Usually defined at beginning of function
- May be defined close to first use

Inner Block Variable Definition in Program 4-29

```
16    if (income >= MIN_INCOME)
17    {
18       // Get the number of years at the current job.
19       cout << "How many years have you worked at "
20            << "your current job? ";
21       int years;     // Variable definition
22       cin >> years;
23
24       if (years > MIN_YEARS)
25          cout << "You qualify.\n";
26       else
27       {
28          cout << "You must have been employed for\n"
29               << "more than " << MIN_YEARS
30               << " years to qualify.\n";
31       }
32    }
```

Variables with the Same Name

- Variables defined inside { } have local or block scope
- When inside a block within another block, can define variables with the same name as in the outer block.
  - When in inner block, outer definition is not available
  - Not a good idea

Two Variables with the Same Name in Program 4-30

Program 4-30

```
1  // This program uses two variables with the name number.
2  #include <iostream>
3  using namespace std;
4
5  int main()
6  {
7     // Define a variable named number.
8     int number;
9
10    cout << "Enter a number greater than 0: ";
11    cin >> number;
12    if (number > 0)
13    {
14       int number;  // Another variable named number.
15       cout << "Now enter another number: ";
16       cin >> number;
17       cout << "The second number you entered was "
18            << number << endl;
19    }
20    cout << "Your first number was " << number << endl;
21    return 0;
22 }
```

Program Output with Example Input Shown in Bold

```
Enter a number greater than 0: 2 [Enter]
Now enter another number: 7 [Enter]
The second number you entered was 7
Your first number was 2
```